Digital Systems

Lecture 4

Assembly Programming

Mark van der Wilk

Hilary Term 2025

UNIVERSITY OF OXFORD

Department of COMPUTER SCIENCE

Following the course of past years by Mike Spivey

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
-
-
-
-

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
- Compiling: Each source (`.c` or `.s`) file becomes an object (`.o`) file.
  - Turning each source file into machine code
  - We saw the machine code of our `adds` and `bx lr` instructions!
  - Unknown absolute memory addresses
-
-
-

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
- Compiling: Each source (`.c` or `.s`) file becomes an object (`.o`) file.
  - Turning each source file into machine code
  - We saw the machine code of our `adds` and `bx lr` instructions!
  - Unknown absolute memory addresses
- Disassembly: From machine code back to assembly.
-
-

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
- Compiling: Each source (`.c` or `.s`) file becomes an object (`.o`) file.
  - Turning each source file into machine code
  - We saw the machine code of our `adds` and `bx lr` instructions!
  - Unknown absolute memory addresses
- Disassembly: From machine code back to assembly.
- Linking: Giving subroutines absolute addresses.
-

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
- Compiling: Each source (`.c` or `.s`) file becomes an object (`.o`) file.
  - Turning each source file into machine code
  - We saw the machine code of our `adds` and `bx lr` instructions!
  - Unknown absolute memory addresses
- Disassembly: From machine code back to assembly.
- Linking: Giving subroutines absolute addresses.
- Startup procedure

Forgot to mention...

What is the format that we actually *send* the final code in?

-
-
-

Forgot to mention...

What is the format that we actually *send* the final code in?

- `more func.hex`

  I.e. *text*-encoded hexadecimal. The "programmer chip" on the micro:bit turns this into binary, before sending it to the Nordic ARM chip.
-
-

Forgot to mention…

What is the format that we actually *send* the final code in?

- `more func.hex`

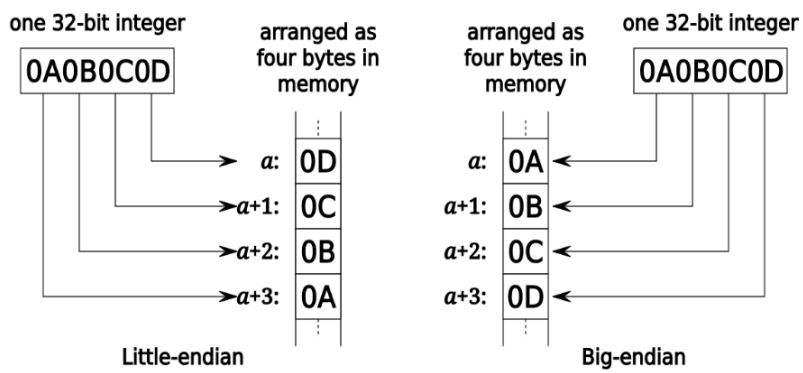
  I.e. *text*-encoded hexadecimal. The "programmer chip" on the micro:bit turns this into binary, before sending it to the Nordic ARM chip.
- Can we still see `18404770` anywhere?
-

Forgot to mention...

What is the format that we actually *send* the final code in?

- `more func.hex`

  I.e. *text*-encoded hexadecimal. The "programmer chip" on the micro:bit turns this into binary, before sending it to the Nordic ARM chip.
- Can we still see `18404770` anywhere?
- No! Instead, we see `40187047`. Stored "little-endian".



If you want to practice + learn more about this:

- Additional questions on this in `problem-sheets/prob1-new.pdf`.
- Answers are included, so good study material.

? Why is there a separation between the compiler and linker?

? Why is there a separation between the compiler and linker?

So you can pre-compile parts of your code.

- Compiling takes time! Some projects take hours to compile.
  Only compiling parts that have changed, saves time.

? Why is there a separation between the compiler and linker?

So you can pre-compile parts of your code.

- Compiling takes time! Some projects take hours to compile.
  Only compiling parts that have changed, saves time.

Simplification and encapsulation of tasks:

- Compiler only needs to know what CPU *core* we're running on, i.e. which instructions our CPU (Cortex-M0) has.
- Linker needs to know details about specific chip, e.g. the memory map.

Digital Systems

Lecture 4

Assembly Programming

Mark van der Wilk

Hilary Term 2025

UNIVERSITY OF OXFORD

Department of COMPUTER SCIENCE

Following the course of past years by Mike Spivey

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
subs r0, r0, r1     @ (a - b) stored in r0
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
subs r0, r0, r1     @ (a - b) stored in r0
muls r0, r3, r0
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
subs r0, r0, r1     @ (a - b) stored in r0
muls r0, r3, r0
bx lr
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
subs r0, r0, r1     @ (a - b) stored in r0
muls r0, r3, r0
bx lr
```

What Instructions Can I Use?

How to look up allowed instructions?

We have three summaries:

- "Rainbow Chart" (included in exam)
- List of common instructions (included in exam)
- Arm Thumb Quick Reference Guide (official from ARM)

Definitive document is ARM v6-M Architecture Reference Manual.

Let's look at details of `add` instruction.

- Index: A.6 Thumb Instruction Details
- A.6.7 Alphabetical list of ARMv6-M Thumb instructions
- Various flavours of `add` instructions.

Different Flavours of Addition

Operation		§	Assembler	Updates	Action	Notes
Add	Immediate 3		`ADDS Rd, Rn, #<imm>`	N Z C V	Rd := Rn + imm	imm range 0-7.
	All registers Lo		`ADDS Rd, Rn, Rm`	N Z C V	Rd := Rn + Rm	
	Hi to Lo, Lo to Hi, Hi to Hi		`ADD Rd, Rd, Rm`		Rd := Rd + Rm	Not Lo to Lo.
	Any to Any	T2	`ADD Rd, Rd, Rm`		Rd := Rd + Rm	Any register to any register.
	Immediate 8		`ADDS Rd, Rd, #<imm>`	N Z C V	Rd := Rd + imm	imm range 0-255.

Optimise The Common Case

- Design principle: Make the common case fast, while leaving uncommon case possible.
- Amdahl's law: The maximum achievable speed-up of an optimisation is limited by how often the optimised part is used.

For the exam:

Familiarise yourself with the instructions, but no need to memorise.

More Assembly Programming

Naive Multiplication in C

```
unsigned func(unsigned a, unsigned b) {
    unsigned x = a, y = b, z = 0;

    /* Invariant: a × b = x × y + z */
    while (y != 0) {
        y = y - 1;
        z = z + x;
    }

    return z;
}
```

Naive Asm Translation of Naive Multiplication in C

```
func:                       @ x in r0, y in r1
    movs r2, #0             @ z = 0
loop:
    cmp r1, #0              @ if y == 0
    beq done                @     jump to done
    subs r1, r1, #1         @ y = y - 1
    adds r2, r2, r0         @ z = z + x
    b loop                  @ jump to loop
done:
    movs r0, r2             @ return z
    bx lr
```

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
-
-
-
-
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
-
-
-
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
-
-
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
- `layout asm`
-
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
- `layout asm`
- `layout regs`
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
- `layout asm`
- `layout regs`
- `break *func`
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
- `layout asm`
- `layout regs`
- `break *func`
- `stepi`

Which Instructions are Actually Used?

```
$ arm-none-eabi-objdump -d mul1.o
00000000 <func>:
   0:  2200      movs    r2, #0

00000002 <loop>:
   2:  2900      cmp     r1, #0
   4:  d002      beq.n   0xc <done>
   6:  3901      subs    r1, #1
   8:  1812      adds    r2, r2, r0
   a:  e7fa      b.n     0x2 <loop>

0000000c <done>:
   c:  0010      movs    r0, r2
   e:  4770      bx      lr
```

Pipelining

- Executing instruction is broken up into stages.
-
-

Pipelining

- Executing instruction is broken up into stages.
- In ARM Cortex-M0, 3 cycles needed to execute a single instruction.
-

Pipelining

- Executing instruction is broken up into stages.
- In ARM Cortex-M0, 3 cycles needed to execute a single instruction.
- However, stages operate in *parallel*, so still 1 instruction *per* cycle.

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

-
-
-
-

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

- When branch is executed, `pc` is 8.
-
-
-

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

- When branch is executed, `pc` is 8.
- Want to jump two instructions ahead.
-
-

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

- When branch is executed, `pc` is 8.
- Want to jump two instructions ahead.
- $\Rightarrow$ Encode offset of 2 instructions!
-

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

- When branch is executed, `pc` is 8.
- Want to jump two instructions ahead.
- $\Rightarrow$ Encode offset of 2 instructions!
- CPU handles multiplication by two (bit shift) before adding to `pc`.

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

-
-
-

-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

-
-
-
-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
-
-
-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
-
-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

Find number of cycles:

-
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

Find number of cycles:

- Can measure (oscilloscope, see labs)
-

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

Find number of cycles:

- Can measure (oscilloscope, see labs)
- Count instructions (7 cycles)

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total), otherwise 1.
- Load / store: Two cycles (one extra for memory access).

Find number of cycles:

- Can measure (oscilloscope, see labs)
- Count instructions (7 cycles)

? Why do branches take 3 cycles?

Pipelining: Normal Operation

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch not taken
6	10	`b`	`adds`	`subs`	next instruction as normal
7	12	`movs`	`b`	`adds`	

Pipelining: Branch Taken

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch taken!
6	12	`movs`	`adds X`	`subs X`	do not execute!
7	14	`bx`	`movs`	`adds X`	do not execute!
8	16	`...`	`bx`	`movs`	as normal

Can We Do Better?

Solutions:

- Better algorithm (e.g. $O(\log n)$).
- "Squeeze" loop.

Optimisation

```
loop:
    cmp r1, #0          @ if y == 0
    beq done            @     jump to done
loop1:
    subs r1, r1, #1     @ y = y - 1
    adds r2, r2, r0     @ z = z + x
    cmp r1, #0          @ if y == 0
    beq done            @     jump to done
    b loop1             @ jump to loop1
done:
```

Optimisation

Swap and simplify.

```
loop:
    cmp r1, #0          @ if y == 0
    beq done            @     jump to done
loop1:
    adds r2, r2, r0     @ z = z + x
    subs r1, r1, #1     @ y = y - 1
    cmp r1, #0          @ if y != 0
    bne loop1           @     jump to loop1
done:
```

Optimisation

Let subs set the condition codes.

```
loop:
    cmp r1, #0              @ if y == 0
    beq done                @     jump to done
loop1:
    adds r2, r2, r0         @ z = z + x
    subs r1, r1, #1         @ y = y - 1
    bne loop1               @ if y != 0, loop
done:
```

- 5 cycles per iteration!
- Would be `do..while` loop in C.

Loop Unrolling

Duplicate the loop body n times. m

```
loop1:
    adds r2, r2, r0
    subs r1, r1, #1
    beq done
    adds r2, r2, r0
    subs r1, r1, #1
    beq done
    ...
```

```
    adds r2, r2, r0
    subs r1, r1, #1
    beq done
    adds r2, r2, r0
    subs r1, r1, #1
    bne loop1
done:
```

- 3.5 cycles per iteration!

Observations on Assembly Programming

- Writing optimised asm code is labourious! Small changes in details of program impact many other lines (e.g. change in register!)
-
-

Debugger is *super* helpful for writing code. Learn it!

Observations on Assembly Programming

- Writing optimised asm code is labourious! Small changes in details of program impact many other lines (e.g. change in register!)
- Good compilers can generate very efficient assembly code from C code.
  - E.g.: It's hard for us to use higher registers. Requires knowing a lot of these quirks of assembly, which instructions can use high registers, and how. Compilers can do this well.
-

Debugger is *super* helpful for writing code. Learn it!

Observations on Assembly Programming

- Writing optimised asm code is labourious! Small changes in details of program impact many other lines (e.g. change in register!)
- Good compilers can generate very efficient assembly code from C code.
  - E.g.: It's hard for us to use higher registers. Requires knowing a lot of these quirks of assembly, which instructions can use high registers, and how. Compilers can do this well.
- It's good to know *how* to write a very fast, tight loop, but no need to spend all your time doing this for *every* bit of code. (Amdahl's law)

Debugger is *super* helpful for writing code. Learn it!

Recap

💪 **Looking up things in datasheets**

💡 **Instruction set restrictions**

... exist for a reason and are specified in the datasheet.

💡 **Pipelining**

- Instructions are executed over multiple clock cycles in parallel.
- Complicates branch instructions, and makes them slower.

💪 **Determining execution time of assembly program**

💪 **More complex assembly programming, and optimisation**